IC526 Advanced Compilers

Lec 18: Optimizations

Yoonseung Kim

DGIST EECS

Fall 2026

Compiler Correctness: Refinement

```
Beh(P) = { observable traces of P }
Beh(target) ⊆ Beh(source)
```

A complete trace can include:

```
  read(5), print(4), normal termination
```

A wrong transformation has a counterexample:
a target trace that the source cannot produce.

Common Subexpression Elimination (CSE)

CSE: Removing Repeated Computations

RustY

```
fn repeated(x: i32,
            y: i32) -> i32 {
  let a: i32 = x + y;
  let b: i32 = x + y;
  return a * b;
}
```

LLVM IR after mem2reg

```
%a = add i32 %x, %y
%b = add i32 %x, %y
%r = mul i32 %a, %b
ret i32 %r
```

After removing unnecessary memory access, redundant computations become visible.

Reuse an Existing Value

Before

```
%a = add i32 %x, %y
%b = add i32 %x, %y
%r = mul i32 %a, %b
ret i32 %r
```

After

```
%a = add i32 %x, %y

%r = mul i32 %a, %a
ret i32 %r
```

Replace the uses of %b with %a, then remove the second add.

Eliminations Can Be Chained

Before

```
%a = add i32 %x, %y   ; a = x + y
%b = add i32 %x, %y   ; b = x + y
%p = mul i32 %a, 3    ; p = a * 3
%q = mul i32 %b, 3    ; q = b * 3
%r = add i32 %p, %q   ; r = p + q
ret i32 %r
```

After

```
%a = add i32 %x, %y   ; a = x + y

%p = mul i32 %a, 3    ; p = a * 3
%q = mul i32 %a, 3    ; q = a * 3
%r = add i32 %p, %q   ; r = p + q
ret i32 %r
```

After replacing %b, the two multiplies have matching operands.

Then, Reuse the Multiply

Before

```
%a = add i32 %x, %y    ; a = x + y

%p = mul i32 %a, 3     ; p = a * 3
%q = mul i32 %a, 3     ; q = a * 3
%r = add i32 %p, %q    ; r = p + q
ret i32 %r
```

After

```
%a = add i32 %x, %y    ; a = x + y

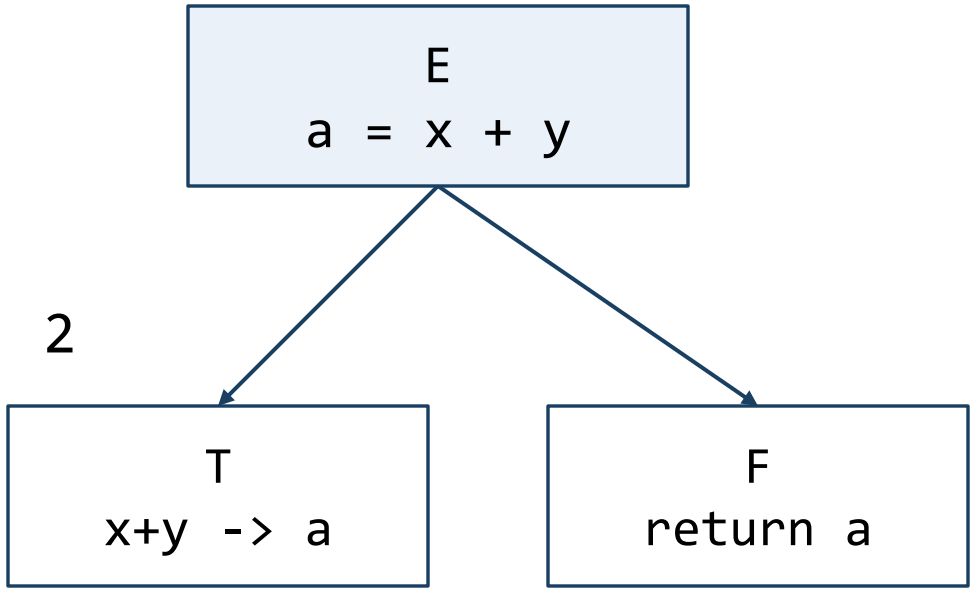
%p = mul i32 %a, 3     ; p = a * 3

%r = add i32 %p, %p    ; r = p + p
ret i32 %r
```

One replacement exposes another redundant computation.

Reuse Across Blocks

```
E:
  %a = add i32 %x, %y
  br i1 %c, label %T, label %F
T:
  %b = add i32 %x, %y
  %r = mul i32 %b, 2    =>   %r = mul i32 %a, 2
  ret i32 %r
F:
  ret i32 %a
```

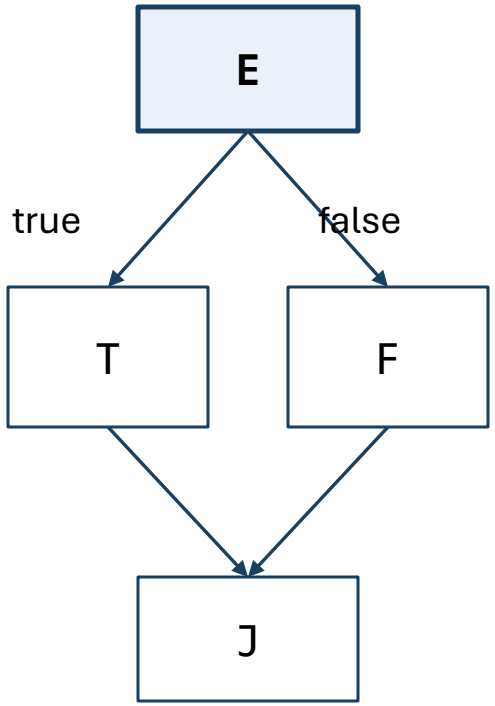


E dominates T, which is sufficient

(why?)

CSE Algorithm: Start at E

CFG



DomTree Preorder:
E → T → F → J

```
E:
  %a = add i32 %x, %y
  br i1 %c, label %T, label %F
T:
  %b = add i32 %x, %y
  %p = mul i32 %b, 3
  br label %J
F:
  %q = mul i32 %x, 3
  br label %J
J:
  %r = phi i32 [%p,%T],[%q,%F]
  %d = add i32 %x, %y
  %w = mul i32 %d, 3
  %v = add i32 %r, %w
  ret i32 %v
```

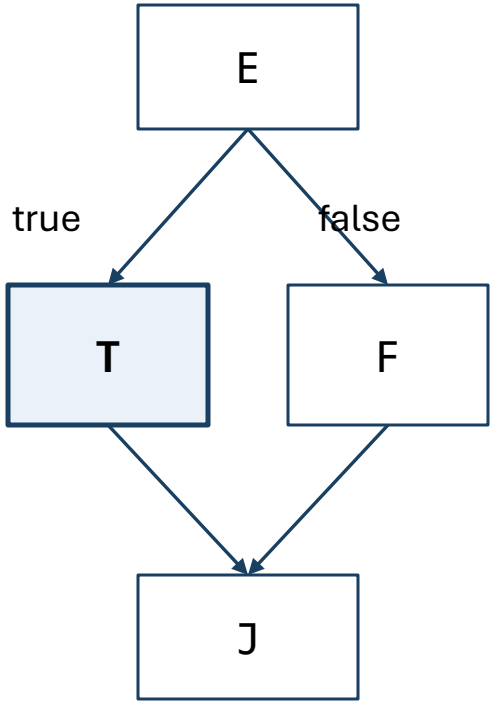
Expression table

Expression	Definitions
add i32 %x, %y	%a (E)

No matching definition.
Keep %a in E.

T: Find a Dominating Definition

CFG




```
E:
  %a = add i32 %x, %y
  br i1 %c, label %T, label %F
T:
  %b = add i32 %x, %y
  %p = mul i32 %b, 3
  br label %J
F:
  %q = mul i32 %x, 3
  br label %J
J:
  %r = phi i32 [%p,%T],[%q,%F]
  %d = add i32 %x, %y
  %w = mul i32 %d, 3
  %v = add i32 %r, %w
  ret i32 %v
```

Expression table

Expression	Definitions
`add i32 %x, %y`	`%a (E)`

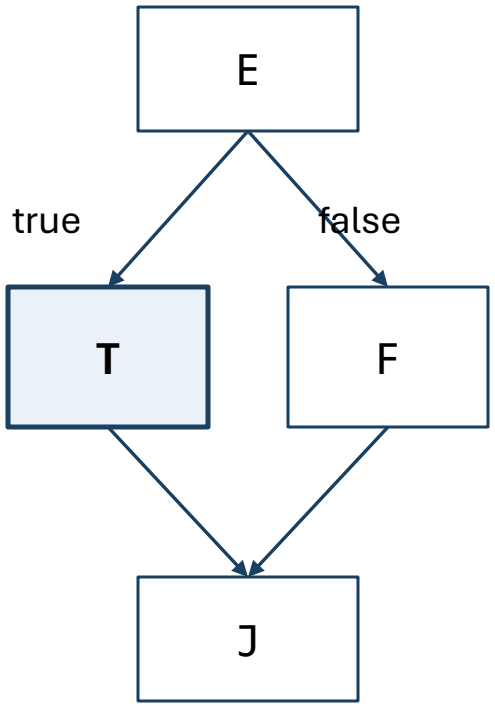
Find %a in E.

E dominates T.

Replace %b with %a.

T: Record the Multiply After Replacement

CFG




```
E:
  %a = add i32 %x, %y
  br i1 %c, label %T, label %F
T:

  %p = mul i32 %a, 3
  br label %J
F:
  %q = mul i32 %x, 3
  br label %J
J:
  %r = phi i32 [%p,%T],[%q,%F]
  %d = add i32 %x, %y
  %w = mul i32 %d, 3
  %v = add i32 %r, %w
  ret i32 %v
```

Expression table

Expression	Definitions
`add i32 %x, %y`	`%a (E)`
`mul i32 %a, 3`	`%p (T)`

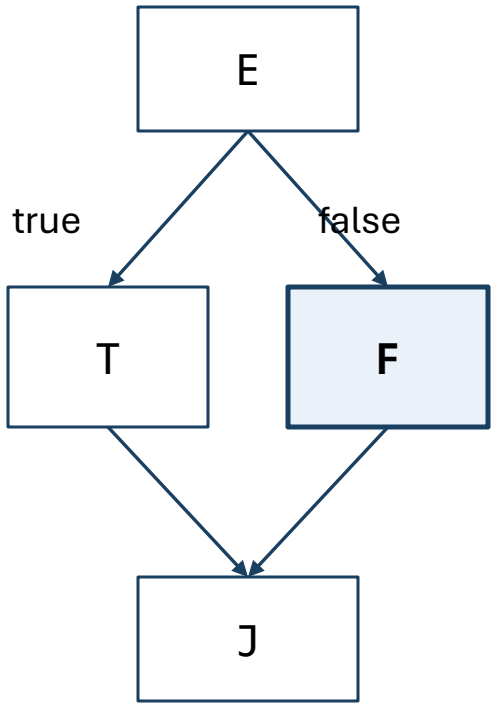
%b is gone.

The multiply now uses %a.

Keep %p in T.

F: Different Operands, Different Expression

CFG




```
E:
  %a = add i32 %x, %y
  br i1 %c, label %T, label %F
T:

  %p = mul i32 %a, 3
  br label %J
F:
  %q = mul i32 %x, 3
  br label %J
J:
  %r = phi i32 [%p,%T],[%q,%F]
  %d = add i32 %x, %y
  %w = mul i32 %d, 3
  %v = add i32 %r, %w
  ret i32 %v
```

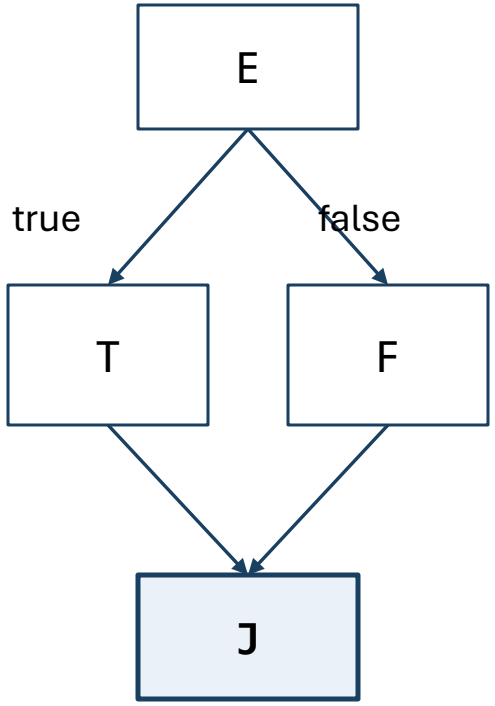
Expression table

Expression	Definitions
`add i32 %x, %y`	`%a (E)`
`mul i32 %a, 3`	`%p (T)`
`mul i32 %x, 3`	`%q (F)`

Keep %q in F.

J: Reuse E's Add Again

CFG




```
E:
  %a = add i32 %x, %y
  br i1 %c, label %T, label %F
T:

  %p = mul i32 %a, 3
  br label %J
F:
  %q = mul i32 %x, 3
  br label %J
J:
  %r = phi i32 [%p,%T],[%q,%F]
  %d = add i32 %x, %y
  %w = mul i32 %d, 3
  %v = add i32 %r, %w
  ret i32 %v
```

Expression table

Expression	Definitions
`add i32 %x, %y`	`%a (E)`
`mul i32 %a, 3`	`%p (T)`
`mul i32 %x, 3`	`%q (F)`

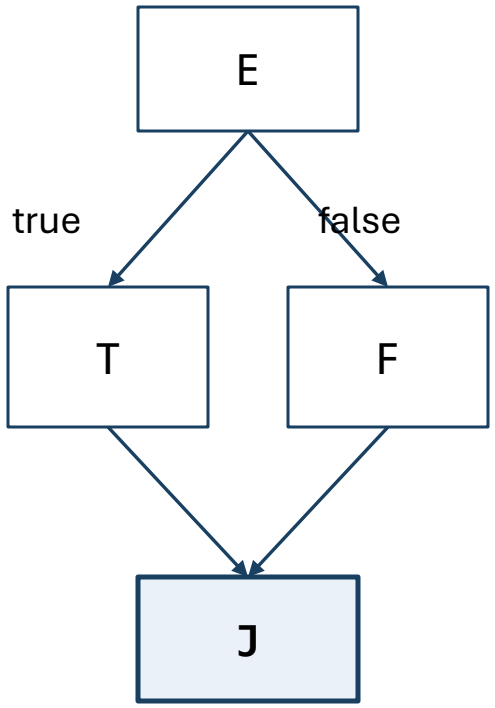
Find %a in E.

E dominates J.

Replace %d with %a.

J: A Matching Definition Cannot Be Reused

CFG




```
E:
  %a = add i32 %x, %y
  br i1 %c, label %T, label %F
T:

  %p = mul i32 %a, 3
  br label %J
F:
  %q = mul i32 %x, 3
  br label %J
J:
  %r = phi i32 [%p,%T],[%q,%F]

  %w = mul i32 %a, 3
  %v = add i32 %r, %w
  ret i32 %v
```

Expression table

Expression	Definitions
`add i32 %x, %y`	`%a (E)`
`mul i32 %a, 3`	`%p(T), %w(J)`
`mul i32 %x, 3`	`%q (F)`

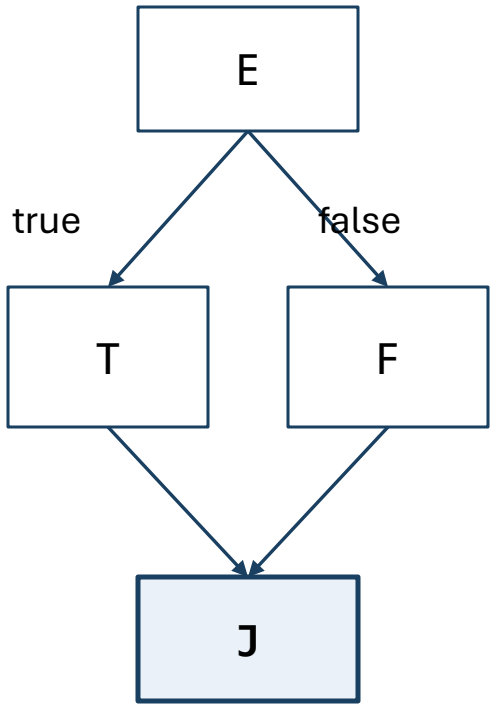
Find %p in T.

T does not dominate J.

Keep and record %w.

J: Finish the Table and the Rewrite

CFG




```
E:
  %a = add i32 %x, %y
  br i1 %c, label %T, label %F
T:

  %p = mul i32 %a, 3
  br label %J
F:
  %q = mul i32 %x, 3
  br label %J
J:
  %r = phi i32 [%p,%T],[%q,%F]

  %w = mul i32 %a, 3
  %v = add i32 %r, %w
  ret i32 %v
```

Expression table

Expression	Definitions
`add i32 %x, %y`	`%a (E)`
`mul i32 %a, 3`	`%p(T), %w(J)`
`mul i32 %x, 3`	`%q (F)`
`add i32 %r, %w`	`%v (J)`

This add is a new expression.
Keep %v in J.
%b and %d are gone.

A Simple CSE Algorithm

```
table = {}
for reachable block B in dominator-tree preorder:
  for each plain add/mul instruction I in B, in order:
    key = (opcode, type, flags, operands)
    R = a definition in table[key] that dominates I
    if R exists:
      replace uses of I with R; erase I
    else:
      table[key].append(I)
```

InstCombine

Example: Combining Two Adds

Before

```
%a = add i32 %x, 5  ; a = x + 5
%r = add i32 %a, 3  ; r = a + 3
```

After

```
%r = add i32 %x, 8  ; r = x + 8
```

Another Example: A Canonical Form

Before

```
%r = mul i32 %x, 8
```

After

```
%r = shl i32 %x, 3
```

The CFG stays the same.

InstCombine: A Large Collection of Small Rewrites

InstCombine contains many rules that combine, simplify, and canonicalize instruction patterns.

```
x + 0                  → x
(x + 5) + 3            → x + 8
x * 8                  → x << 3
(x & 15) & 3           → x & 3
select c, x, x         → x
```

Rules cover arithmetic, bit operations, comparisons, casts, and more.

Each rule handles a small pattern.

InstCombine: Can the Division Cancel?

Before

```
x = read();   // i32
q = x / 2;
r = q * 2;
print(r);
```

```
read(5)
print(4)
normal termination
```

Incorrect target

```
x = read();
print(x);
```

```
read(5)
print(5)
normal termination
```

Beh(target) ⊈ Beh(source)

InstCombine: Can the Division Cancel?

Before

```
x = read();    // i32
q = 1 * x;
r = q / x;
print(r);
```

```
read(0)
UB
```

Correct target

```
x = read();
print(1);
```

```
read(0)
print(1)
normal termination
```

Beh(target) ⊆ Beh(source)

Loop-Invariant Code Motion (LICM)

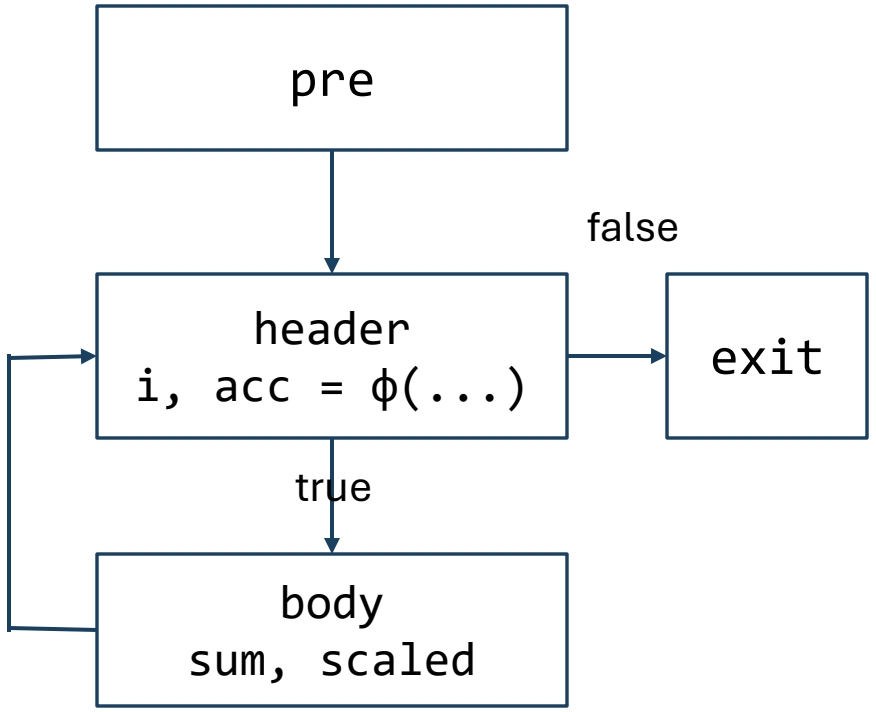
LICM: Repeated Work in a Loop

```
fn accumulate(n: i32, a: i32, b: i32) -> i32 {
  let mut i: i32 = 0;
  let mut acc: i32 = 0;
  while i < n {
    acc = acc + (a + b) * 3;
    i = i + 1;
  }
  return acc;
}
```

Which computations depend on the iteration?

The Initial Loop

```
pre:
  br label %header
header:
  %i = phi i32 [0,%pre],[%next,%body]
  %acc = phi i32 [0,%pre],[%acc.next,%body]
  %more = icmp slt i32 %i, %n
  br i1 %more, label %body, label %exit
body:
  %sum = add i32 %a, %b
  %scaled = mul i32 %sum, 3
  %acc.next = add i32 %acc, %scaled
  %next = add i32 %i, 1
  br label %header
exit:
  ret i32 %acc
```



All operands are outside the loop

Move the Add to the Preheader

```
pre:
  %sum = add i32 %a, %b
  br label %header
header:
  %i = phi i32 [0,%pre],[%next,%body]
  %acc = phi i32 [0,%pre],[%acc.next,%body]
  %more = icmp slt i32 %i, %n
  br i1 %more, label %body, label %exit
body:
  %scaled = mul i32 %sum, 3
  %acc.next = add i32 %acc, %scaled
  %next = add i32 %i, 1
  br label %header
exit:
  ret i32 %acc
```

Moved to pre

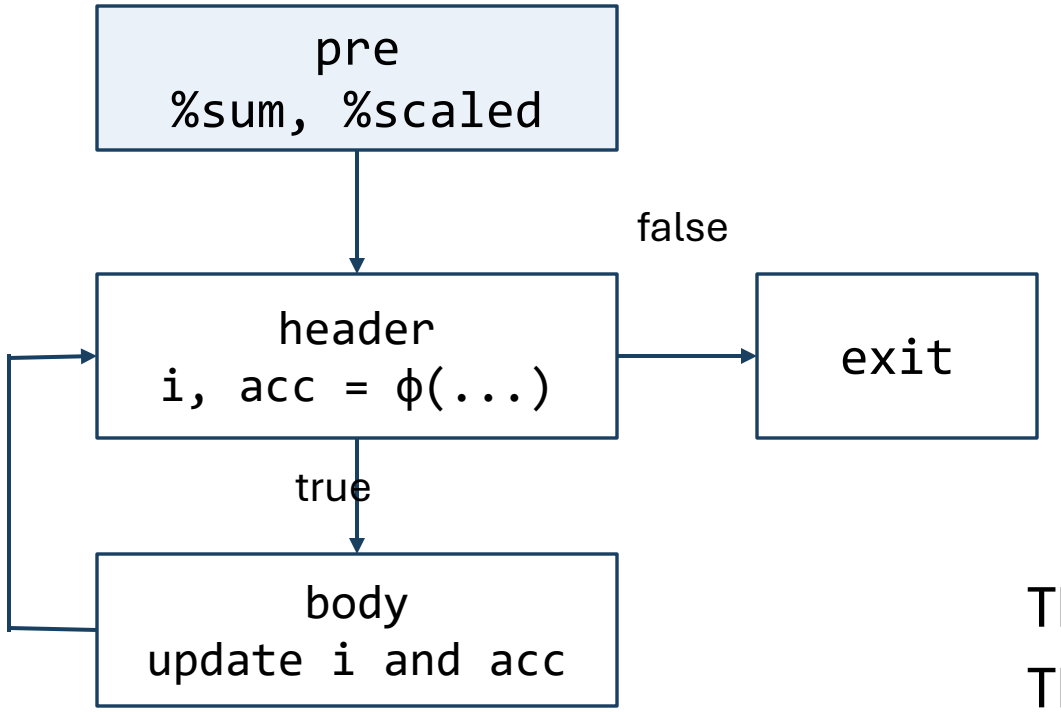
Then Move the Multiply

```
pre:
  %sum = add i32 %a, %b
  %scaled = mul i32 %sum, 3
  br label %header
header:
  %i = phi i32 [0,%pre],[%next,%body]
  %acc = phi i32 [0,%pre],[%acc.next,%body]
  %more = icmp slt i32 %i, %n
  br i1 %more, label %body, label %exit
body:
  %acc.next = add i32 %acc, %scaled
  %next = add i32 %i, 1
  br label %header
exit:
  ret i32 %acc
```

Moved to pre

“Invariant” Does Not Mean Safe to Hoist

Compute once in pre.



n = 0

The body is skipped.

The hoisted code still runs.

“Invariant” Does Not Mean Safe to Hoist

Before

```
n = read();
i = 0;
while i < n {
  q = 10 / n;
  print(q);
  i = i + 1;
}
```

Incorrect hoist

```
n = read();
q = 10 / n;
i = 0;
while i < n {
  print(q);
  i = i + 1;
}
```

A target trace that the source cannot produce:

`read(0), print(99), normal termination  (by UB)`

Beh(target) ⊈ Beh(source)

The Correctness Condition for All

Optimization	What changes?
CSE	Reuse an existing result
InstCombine	Rewrite an instruction pattern
LICM	Move a computation out of a loop

Beh(target) ⊆ Beh(source)

Reads, prints, memory effects, UB, and termination matter.

Next: From LLVM IR to an Executable

LLVM IR
- → machine instructions
- → physical registers and stack slots
- → object file
- → executable